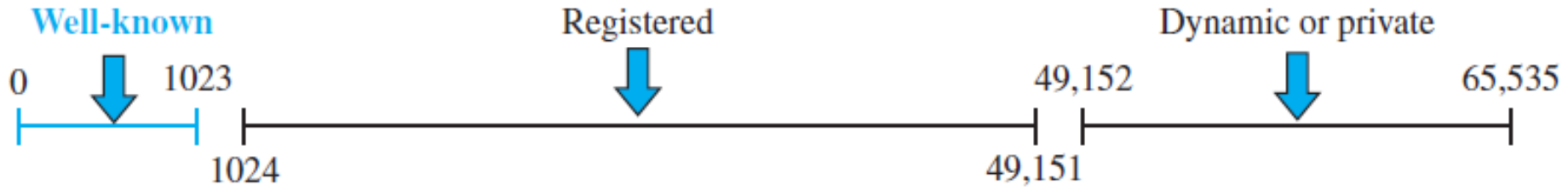
Transport Layer

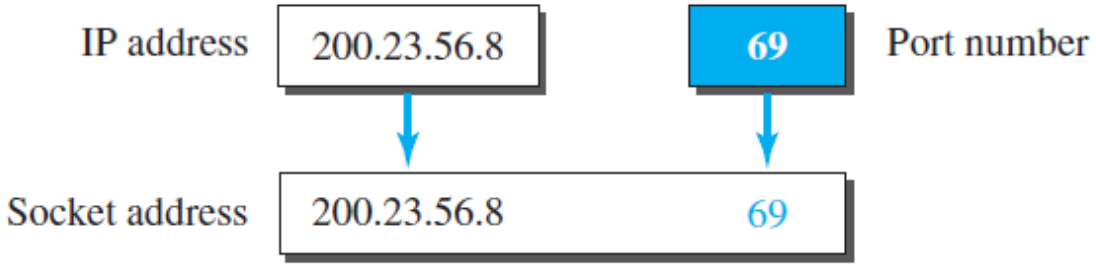
Basics

- **Process-to-process communication** between two application layers
  - A process is an application-layer entity (running program) that uses the services of the transport layer
- ***Addressing: Port Numbers***
  - the most common is through the **client-server paradigm**
  - For communication, we must define the local host, local process, remote host, and remote process.
  - To define the processes, we need second identifiers, called ***port numbers.*** In the TCP/IP protocol suite, the port numbers are integers between 0 and 65,535

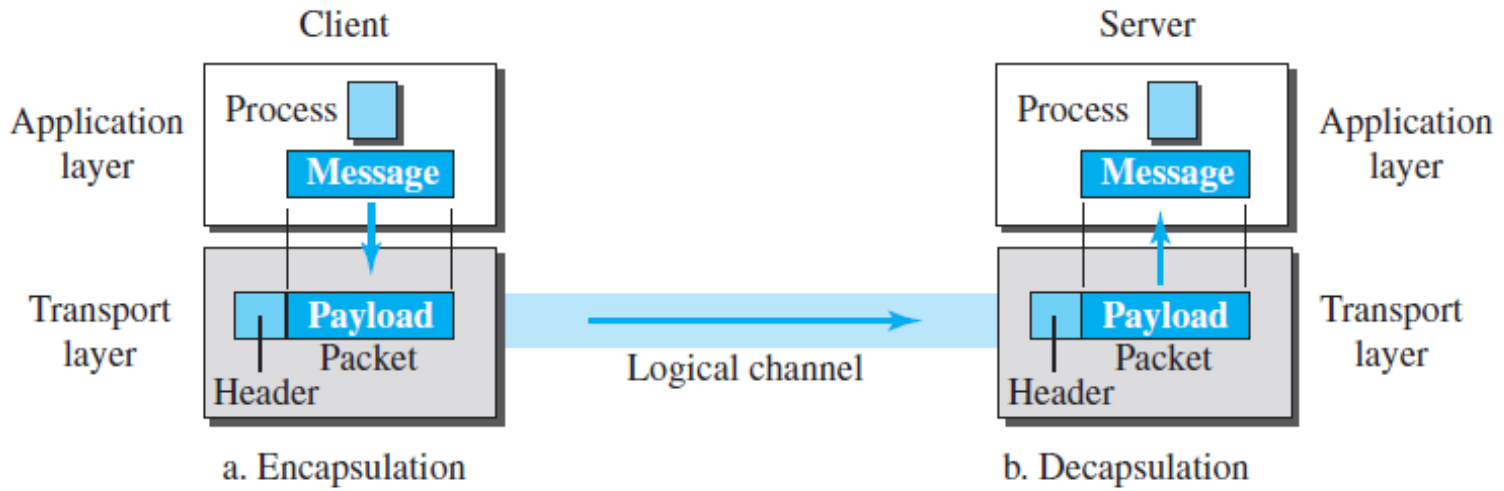
- ***ICANN Ranges***
- divided the port numbers into three ranges: well-known, registered, and dynamic (or private)



- ***Socket Addresses***
  - The combination of an IP address and a port number is called a ***socket address***



- ***Encapsulation and Decapsulation***
  - Encapsulation happens at the sender site. Decapsulation happens at the receiver site



- ***Multiplexing and Demultiplexing***
  - Whenever an entity accepts items from more than one source, this is referred to as ***multiplexing*** (many to one).
  - Whenever an entity delivers items to more than one source, this is referred to as ***demultiplexing*** (one to many).
- ***Sequence Numbers***
  - When a packet is corrupted or lost, the receiving transport layer can somehow inform the sending transport layer to resend that packet using the sequence number.
  - The receiving transport layer can also detect duplicate packets if two received packets have the same sequence number.
  - The out-of-order packets can be recognized by observing gaps in the sequence numbers.
  - Packets are numbered sequentially.
  - If the header of the packet allows m bits for the sequence number, the sequence numbers range from 0 to $2^m - 1$.

- if *m* is 4, the only sequence numbers are 0 through 15, inclusive

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...

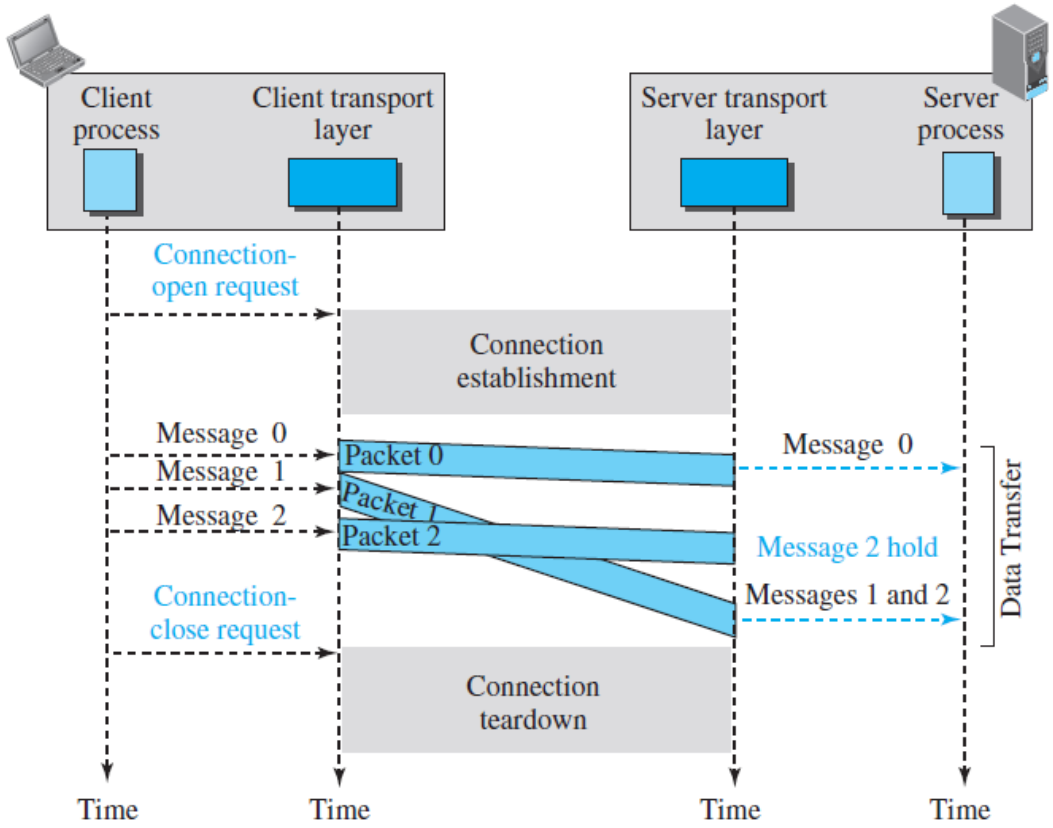
- ***Acknowledgment***
  - Receiver side, sends an ack for correct packets
  - discard the corrupted packets
  - detect lost packets if it uses a timer.
  - Duplicate packets can be silently discarded by the receiver
  - Out-of-order packets can be either discarded (to be treated as lost packets by the sender), or stored until the missing one arrives.
- ***Sliding Window***

a. Four packets have been sent.

- ***Congestion Control***
  - Congestion in a network may occur if the *load* on the network—the number of packets sent to the network—is greater than the *capacity* of the network—the number of packets a network can handle.
  - **Congestion control** refers to the mechanisms and techniques that control the congestion and keep the load below the capacity.
  - Congestion in a network or internetwork occurs because routers and switches have queues—buffers that hold the packets before and after processing.
  - input queue and an output queue for each interface - If a router cannot process the packets at the same rate at which they arrive, the queues become overloaded and congestion occurs.

Connectionless and Connection-Oriented Protocols

- ***Connectionless Service***
  - independency between packets
- ***Connection-Oriented Service***
  - the client and the server first need to establish a logical connection between themselves



Services

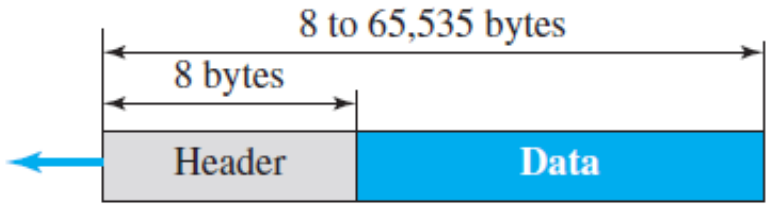
- ***UDP***
  - UDP is an unreliable connectionless transport-layer protocol.
- ***TCP***
  - TCP is a reliable connection-oriented protocol
- ***SCTP***
  - SCTP is a new transport-layer protocol that combines the features of UDP and TCP.

User Datagram Protocol (UDP)

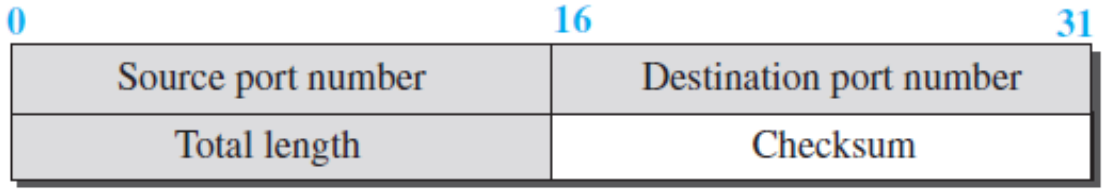
- UDP is a very simple protocol using a minimum of overhead.
- If a process wants to send a small message and does not care much about reliability, it can use UDP.
- Sending a small message using UDP takes much less interaction between the sender and receiver than using TCP.

User Datagram

- UDP packets, called ***user datagrams,*** have a fixed-size header of 8 bytes made of four fields, each of 2 bytes (16 bits).



a. UDP user datagram



b. Header format

- The following is the content of a UDP header in hexadecimal format.

CB84000D001C001C

a. What is the source port number?

b. What is the destination port number?

c. What is the total length of the user datagram?

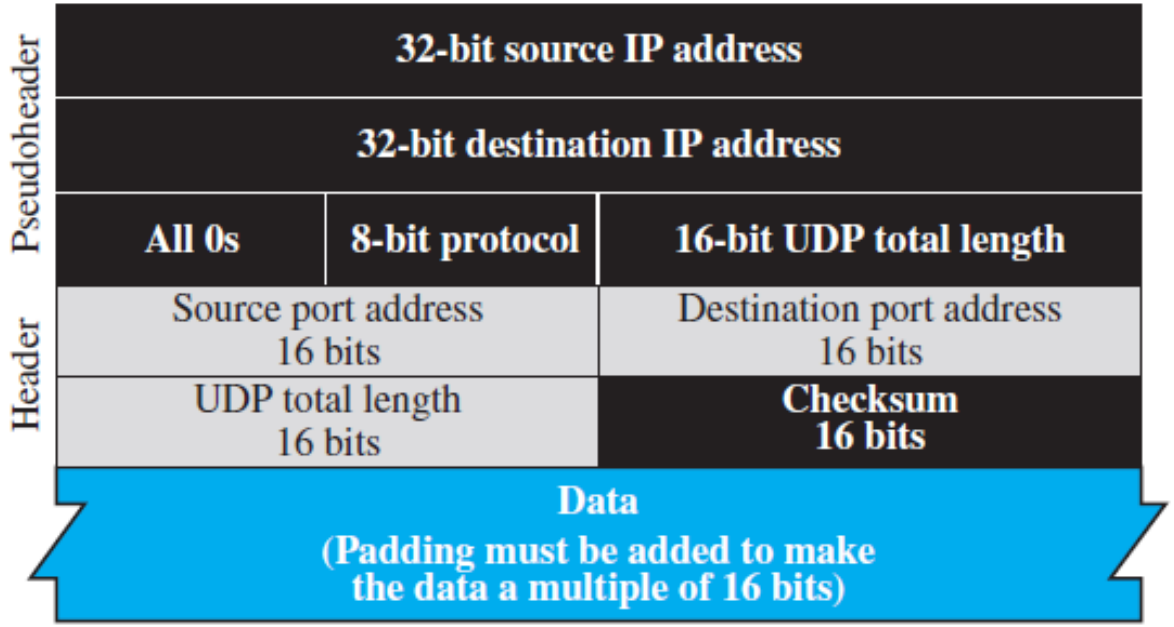
d. What is the length of the data?

e. Is the packet directed from a client to a server or vice versa?

f. What is the client process?

- The source port number is the first four hexadecimal digits $(CB84)_{16}$, which means that the source port number is 52100.
- The destination port number is the second four hexadecimal digits $(000D)_{16}$, which means that the destination port number is 13.
- The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
- The length of the data is the length of the whole packet minus the length of the header, or 28 − 8 = 20 bytes.
- Since the destination port number is 13 (well-known port), the packet is from the client to the server.
- The client process is the Daytime

- ***Checksum***
- UDP checksum calculation includes three sections: a pseudo header, the UDP header, and the data coming from the application layer



Transmission Control Protocol

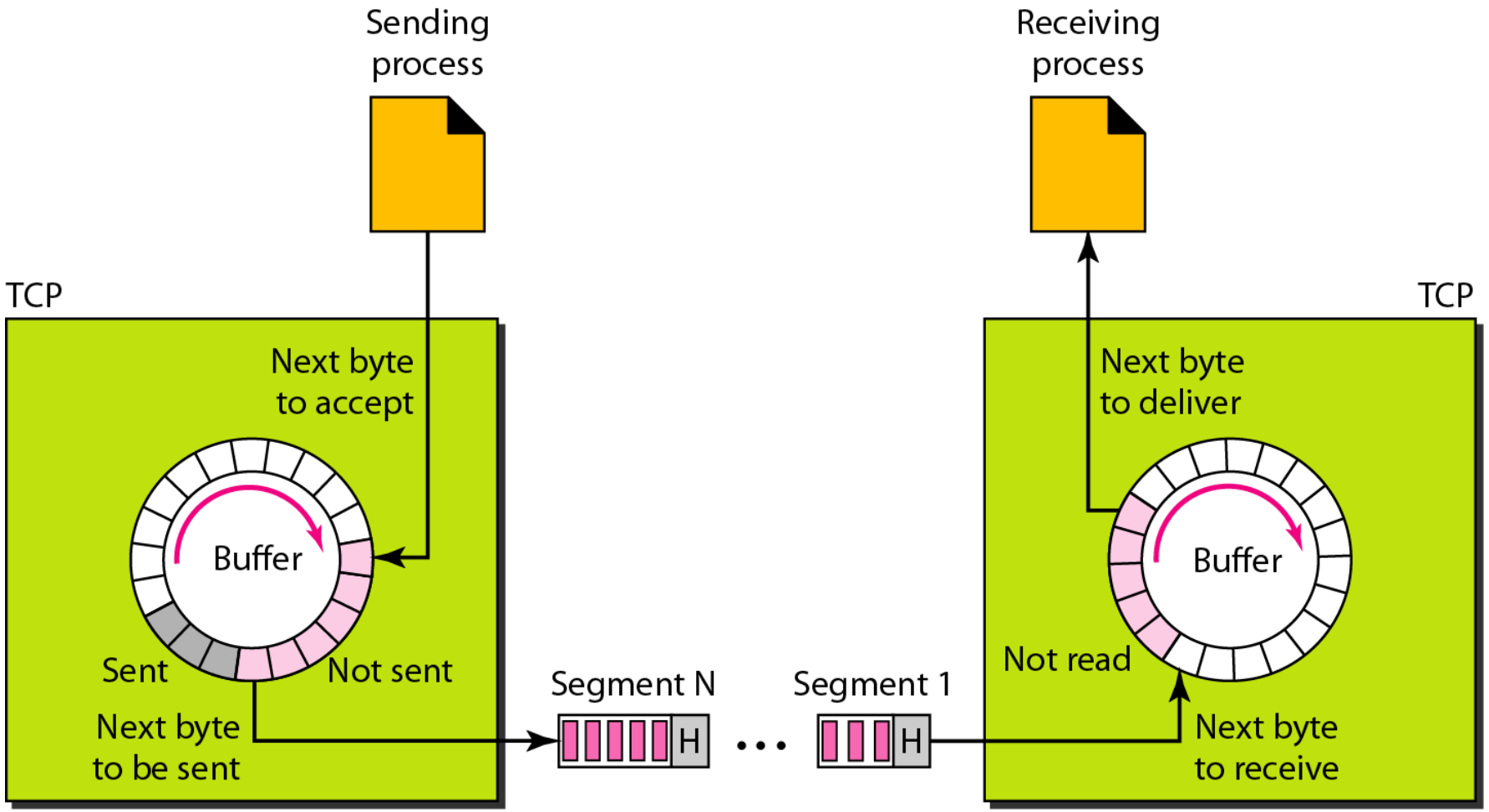
- process-to-process (program-to-program) protocol.
- uses port numbers
- TCP is a connection oriented protocol; it creates a virtual connection between two TCPs to send data.
- uses flow and error control mechanisms at the transport level

TCP Services

- *Process-to-Process Communication*

Port	*Protocol*	*Description*
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

- *Stream Delivery Service*
  - allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes.
  - Sending and Receiving Buffers
    - sending and the receiving processes may not write or read data at the same speed, TCP needs buffers for storage
    - circular array of I-byte locations


Sending process
Receiving process
TCP
TCP
Next byte to accept
Next byte to deliver
Buffer
Buffer
Sent
Not sent
Not read
Segment N
Segment 1
H
H
Next byte to be sent
Next byte to receive

Segments

- TCP groups a number of bytes together into a packet called a segment.
- segments are encapsulated in IP datagrams and transmitted

TCP Features

- ***Numbering System***
  - Byte Number
    - When TCP receives bytes of data from a process, it stores them in the sending buffer and numbers them.
    - TCP generates a random number between 0 and $2^{32} - 1$.
    - if the random number happens to be 1057 and the total data to be sent are 6000 bytes.

    the bytes are numbered from **1057 to 7056**

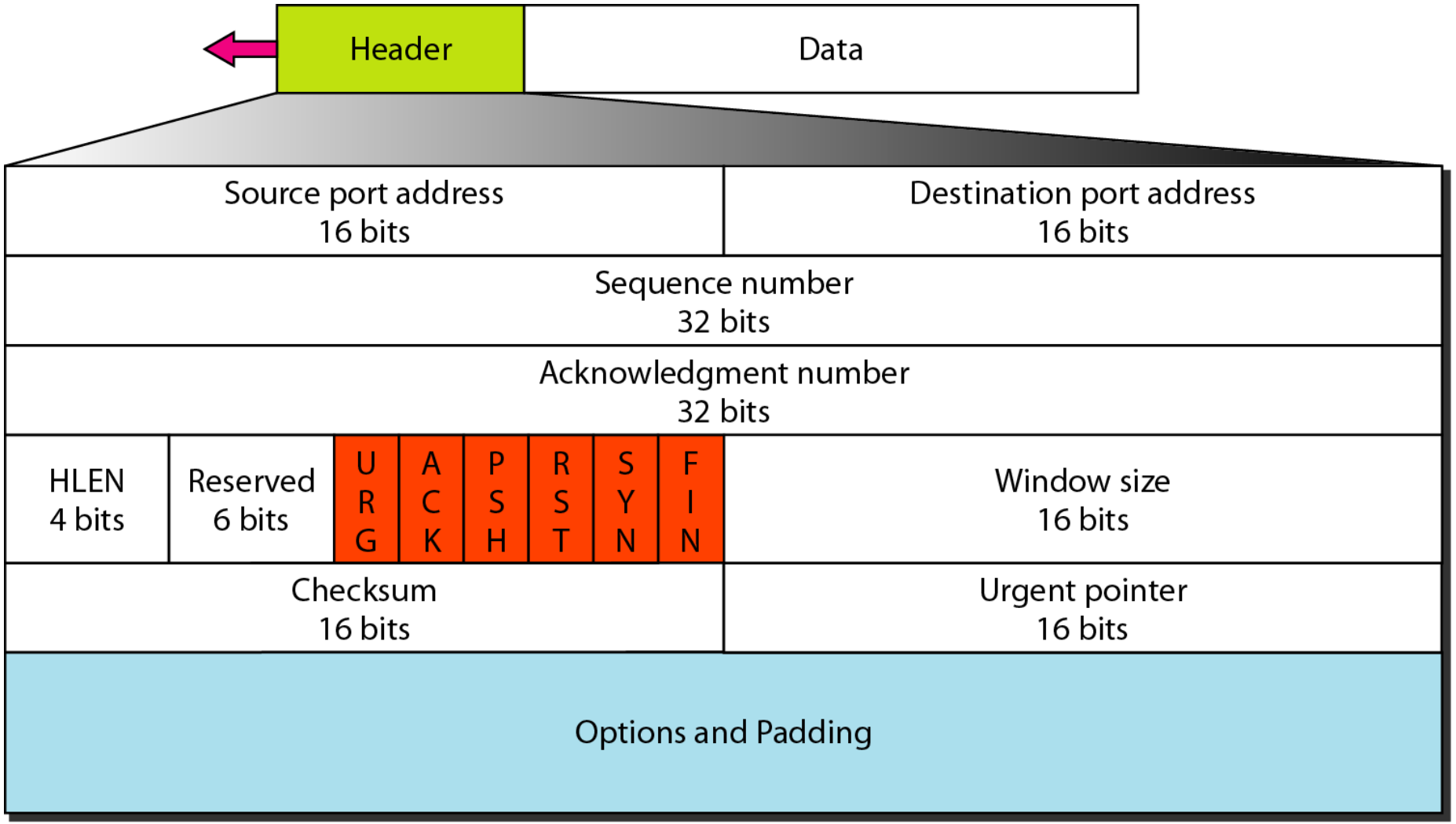
- **Sequence Number**
  - TCP assigns a sequence number to each segment that is being sent.
  - The sequence number for each segment is the number of the first byte carried in that segment.
- Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10,001 What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

Segment 1	→	Sequence Number: 10,001 (range: 10,001 to 11,000)
Segment 2	→	Sequence Number: 11,001 (range: 11,001 to 12,000)
Segment 3	→	Sequence Number: 12,001 (range: 12,001 to 13,000)
Segment 4	→	Sequence Number: 13,001 (range: 13,001 to 14,000)
Segment 5	→	Sequence Number: 14,001 (range: 14,001 to 15,000)

- **Acknowledgment Number**
  - sequence number in each direction shows the number of the first byte carried by the segment
  - Each party also uses an acknowledgment number to confirm the bytes it has received.
  - the party takes the number of the last byte that it has received, safe and sound, adds I to it, and announces this sum as the acknowledgment number.

- ***Flow Control and Error control***
  - The receiver of the data controls the amount of data that are to be sent by the sender.
  - To provide reliable service, TCP implements an error control mechanism
- ***Congestion Control***
  - The amount of data sent by a sender is not only controlled by the receiver (flow control), but is also determined by the level of congestion in the network.

TCP Data Format



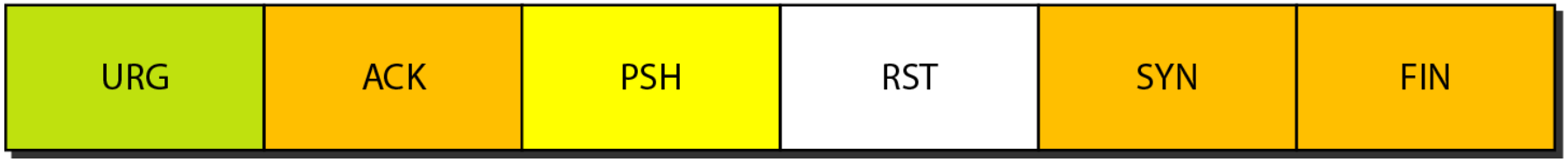
- segment consists of a 20- to 60-byte header (20 bytes if there are no options and up to 60 bytes if it contains options)
- **Source port address.**
  - 16-bit field that defines the port number of the application program in the host that is sending the segment
- **Destination port address.**
  - 16-bit field that defines the port number of the application program in the host that is receiving the segment

- **Sequence number.**
  - 32-bit field defines the number assigned to the first byte of data contained in this segment.
- **Acknowledgment number**
  - 32-bit field defines the byte number that the receiver of the segment is expecting to receive from the other party

- **Header length**
  - 4-bit field indicates the number of 4-byte words in the TCP header
  - Therefore, the value of this field can be between 5 (5 x 4 =20) and 15 (15 x 4 =60).
  - It helps in knowing from where the actual data begins.
- **Reserved**. This is a 6-bit field reserved for future use.
- **Control**.
  - 6 different control bits or flags

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

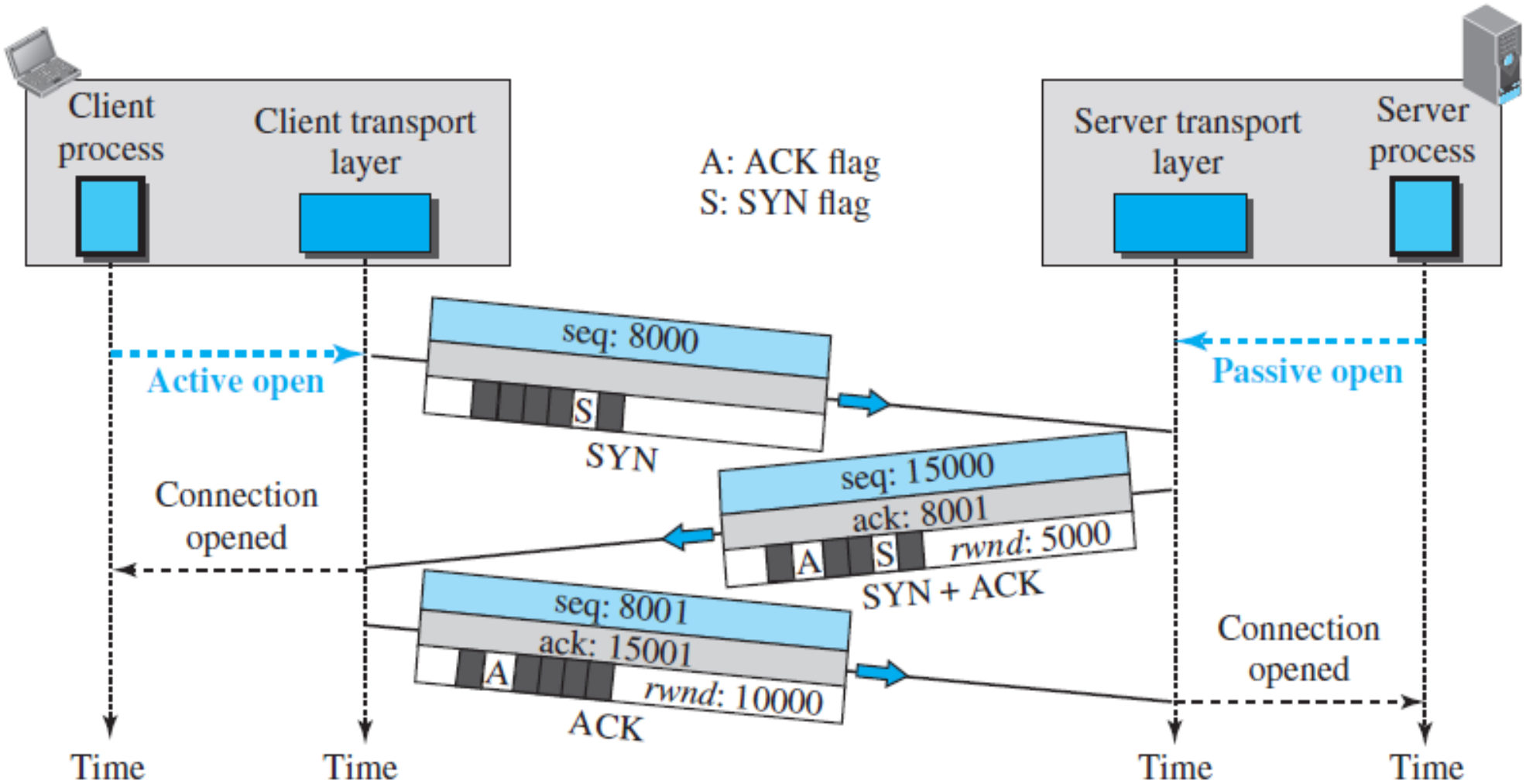
RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



- URG bit is used to treat certain data on an urgent basis.
  - The urgent data has be prioritized.
  - Receiver forwards urgent data to the receiving application on a separate channel.
- ACK bit indicates whether acknowledgement number field is valid or not.
  - For all TCP segments except request segment, ACK bit is set to 1.
  - Request segment is sent for connection establishment during **Three Way Handshake**.

- PSH bit is used to push the entire buffer immediately to the receiving application.
  - This makes the entire buffer to free up immediately
- RST bit is used to reset the TCP connection.
  - It causes both the sides to release the connection and all its resources abnormally.
- SYN bit is used to synchronize the sequence numbers.
  - It indicates the receiver that the sequence number contained in the TCP header is the initial sequence number.
  - Request segment sent for connection establishment during Three way handshake contains SYN bit set to 1.
- FIN bit is used to terminate the TCP connection.
  - It indicates the receiver that the sender wants to terminate the connection.

- Window size
  - defines the size of the window, in bytes, that the other party must maintain.
  - The window size changes dynamically during data transmission.
  - It usually increases during TCP transmission up to a point where congestion is detected.
  - After congestion is detected, the window size is reduced to avoid having to drop packets.
- Checksum.
  - This 16-bit field contains the checksum
- Urgent pointer
  - This l6-bit field, which is valid only if the urgent flag is set, is used when the segment contains urgent data
  - It indicates how much data in the current segment counting from the first data byte is urgent.
  - Urgent pointer added to the sequence number indicates the end of urgent data byte.
- Options.
  - There can be up to 40 bytes of optional information in the TCP header

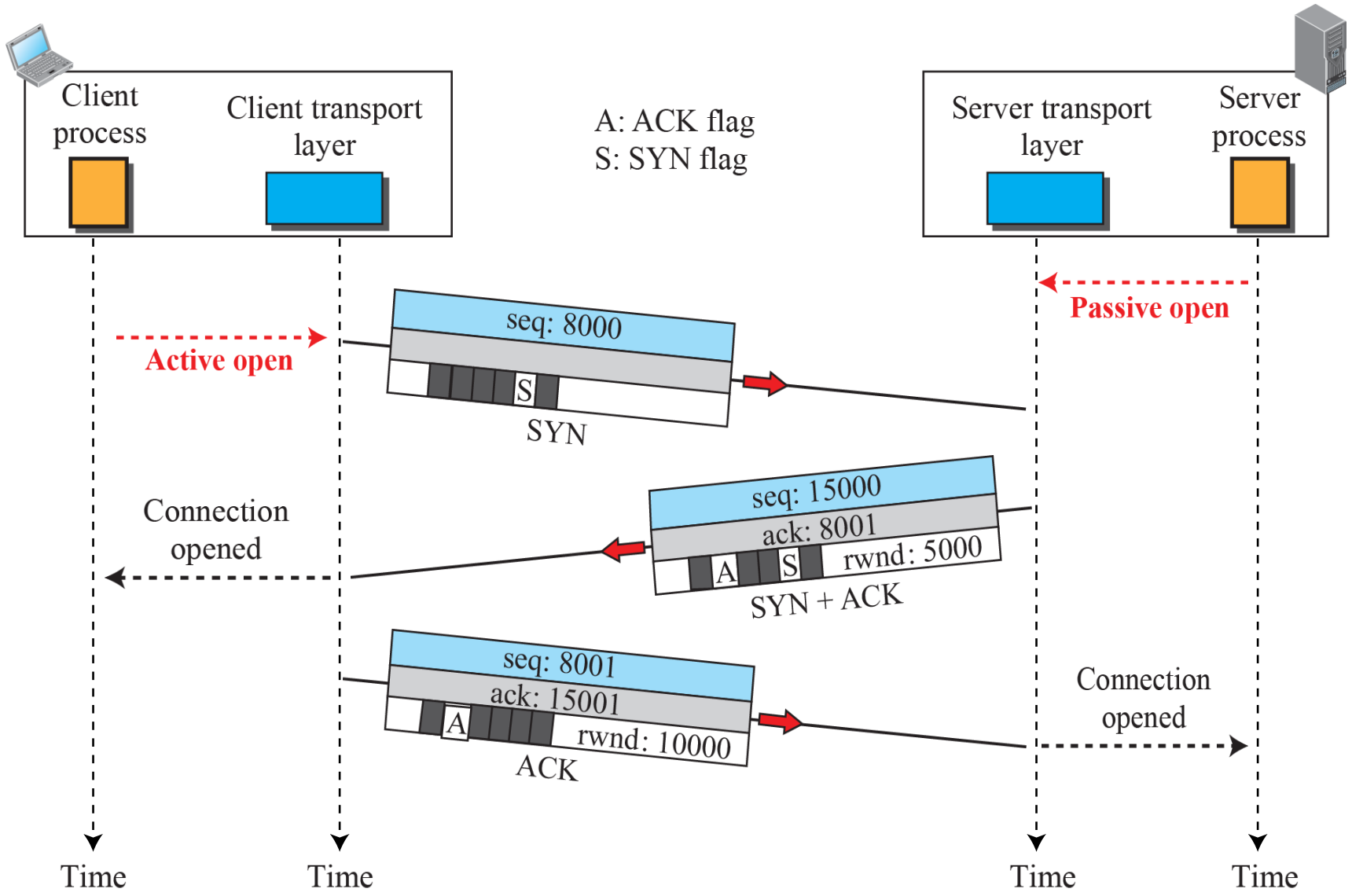

Client process
Client transport layer
A: ACK flag
S: SYN flag
Server transport layer
Server process
Active open
seq: 8000
S
SYN
Passive open
Connection opened
seq: 15000
ack: 8001
A
S
rwnd: 5000
SYN + ACK
seq: 8001
ack: 15001
A
rwnd: 10000
ACK
Connection opened
Time
Time
Time
Time

- ***SYN Flooding Attack***
  - when one or more malicious attackers send a large number of SYN segments to a server pretending that each of them is coming from a different client by faking the source IP addresses in the datagrams.
  - Denial of service attack
  - Remedies – maintain cookies.

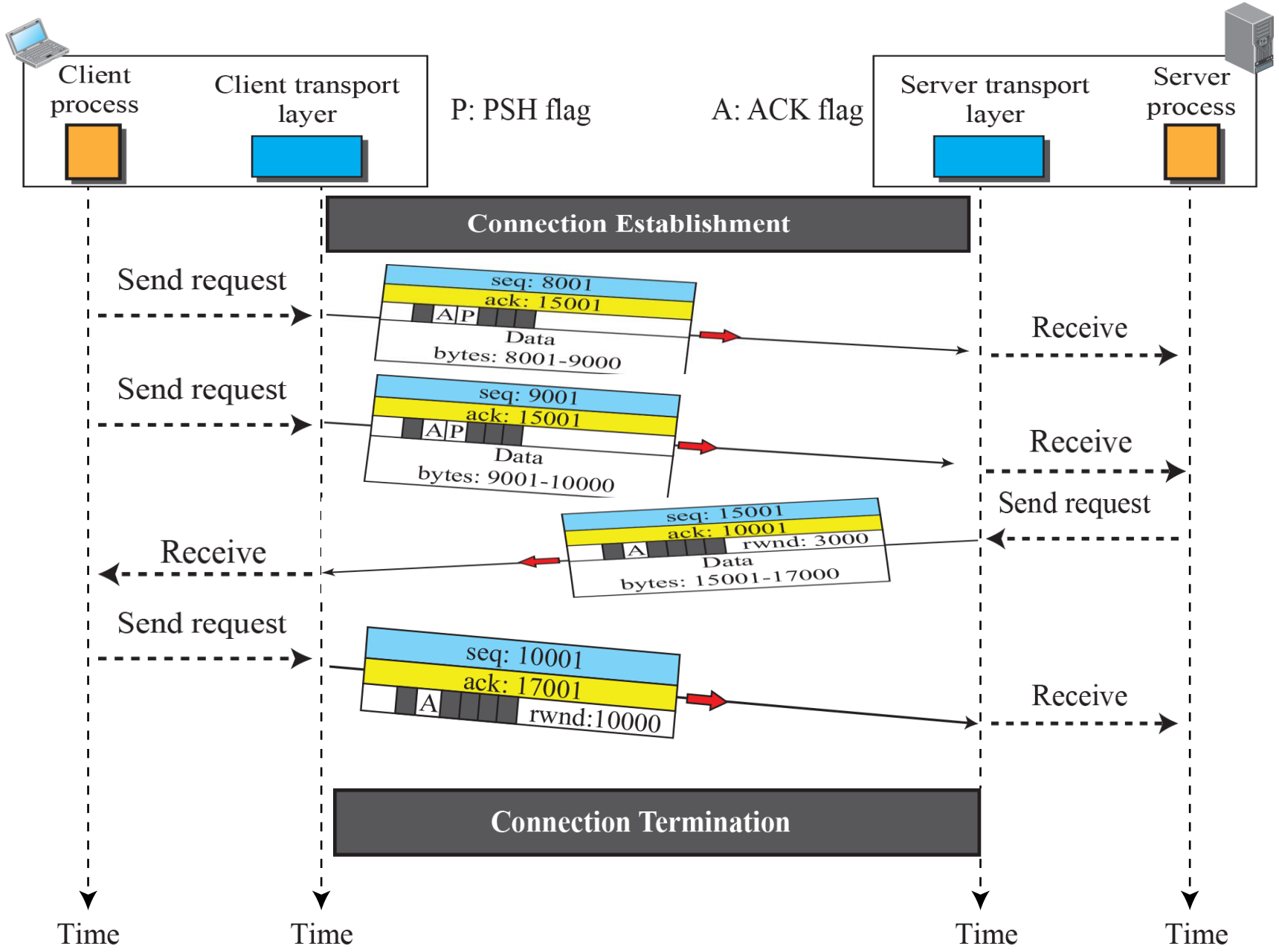
TCP Connection Establishment

TCP is connection-oriented. All of the segments belonging to a message are then sent over this logical path. Using a single logical pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames. You may wonder how TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented. The point is that a TCP connection is logical, not physical. TCP operates at a higher level. TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself.

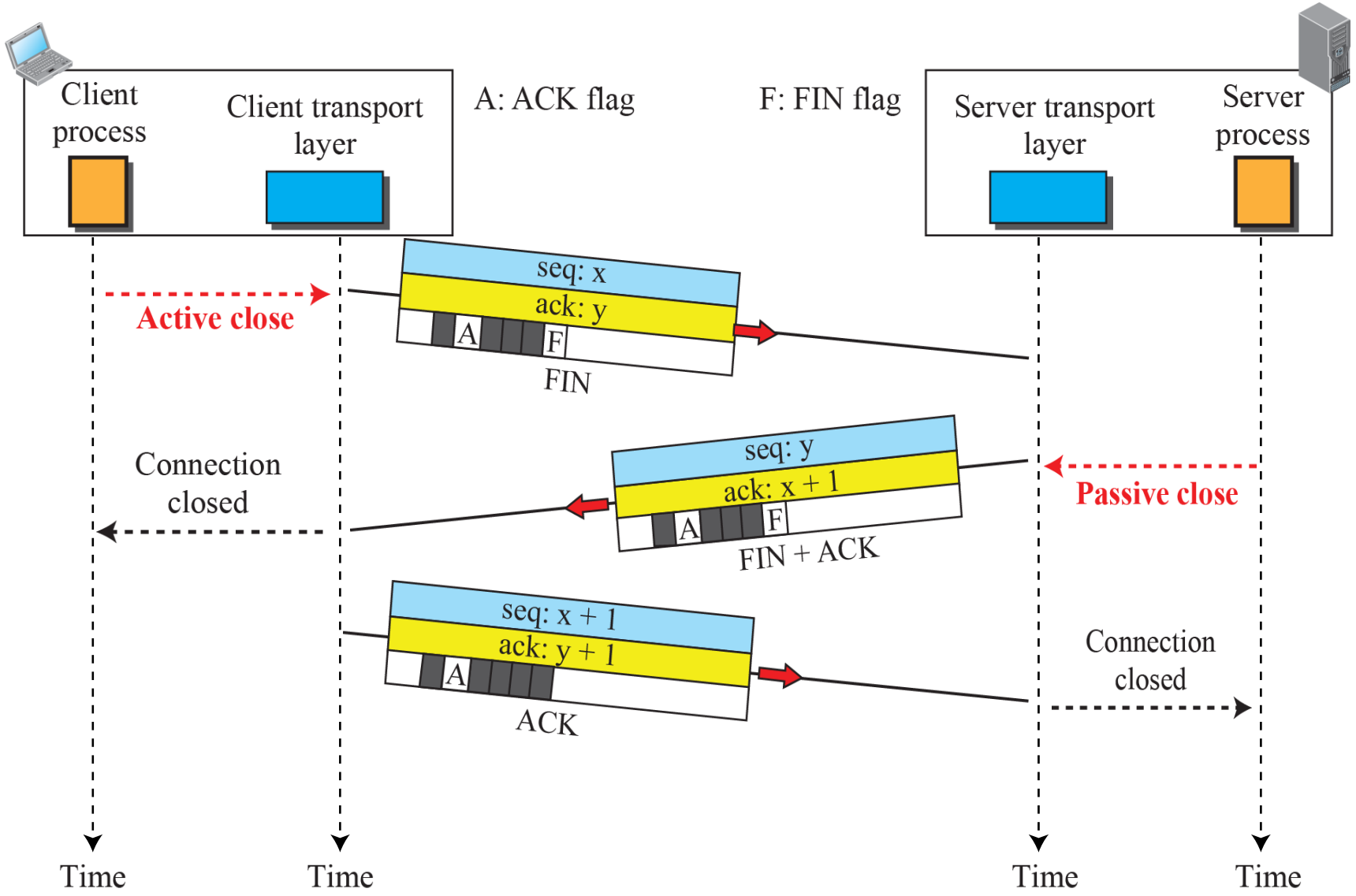
Connection establishment using three-way handshaking



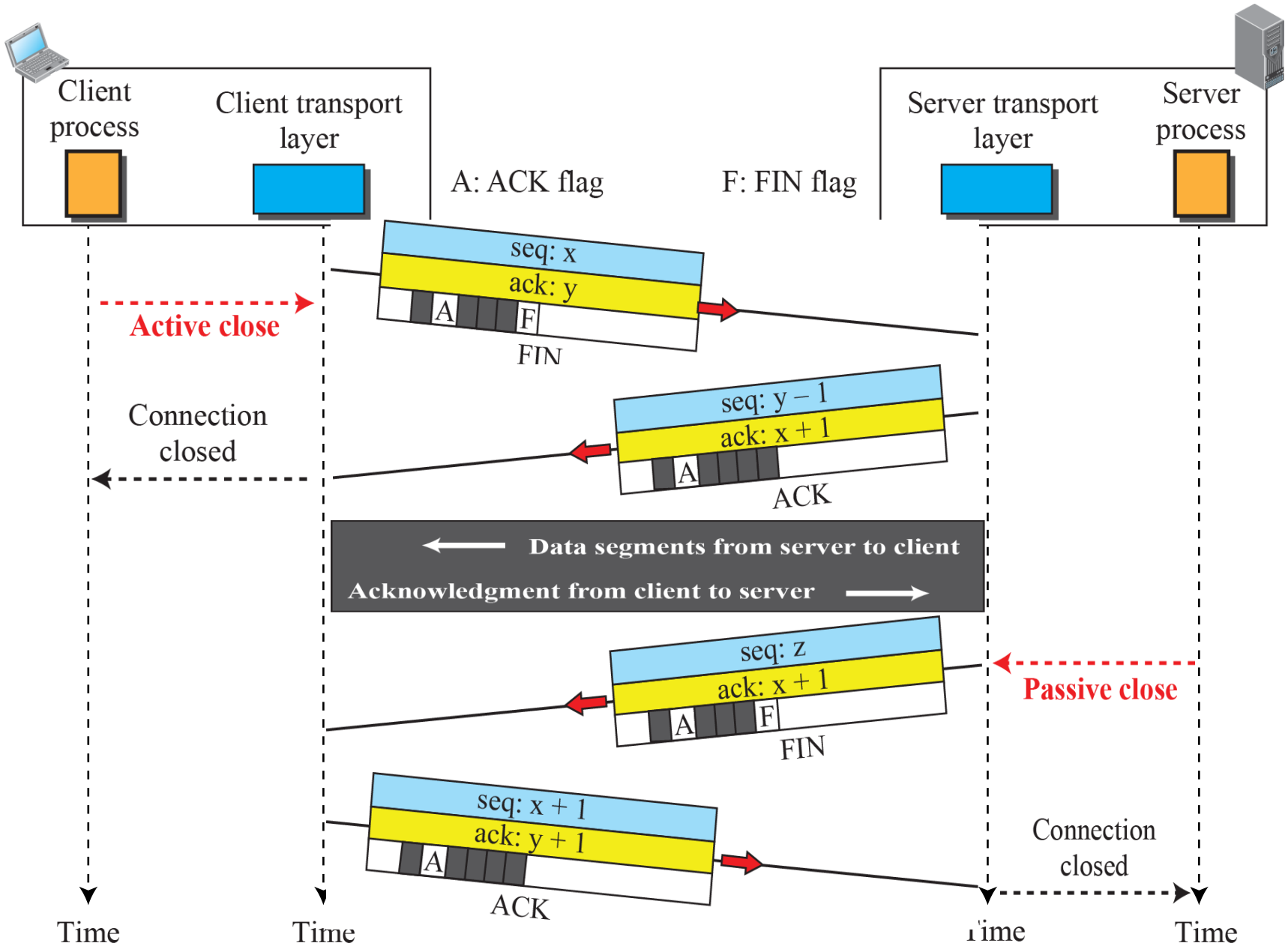
Data transfer



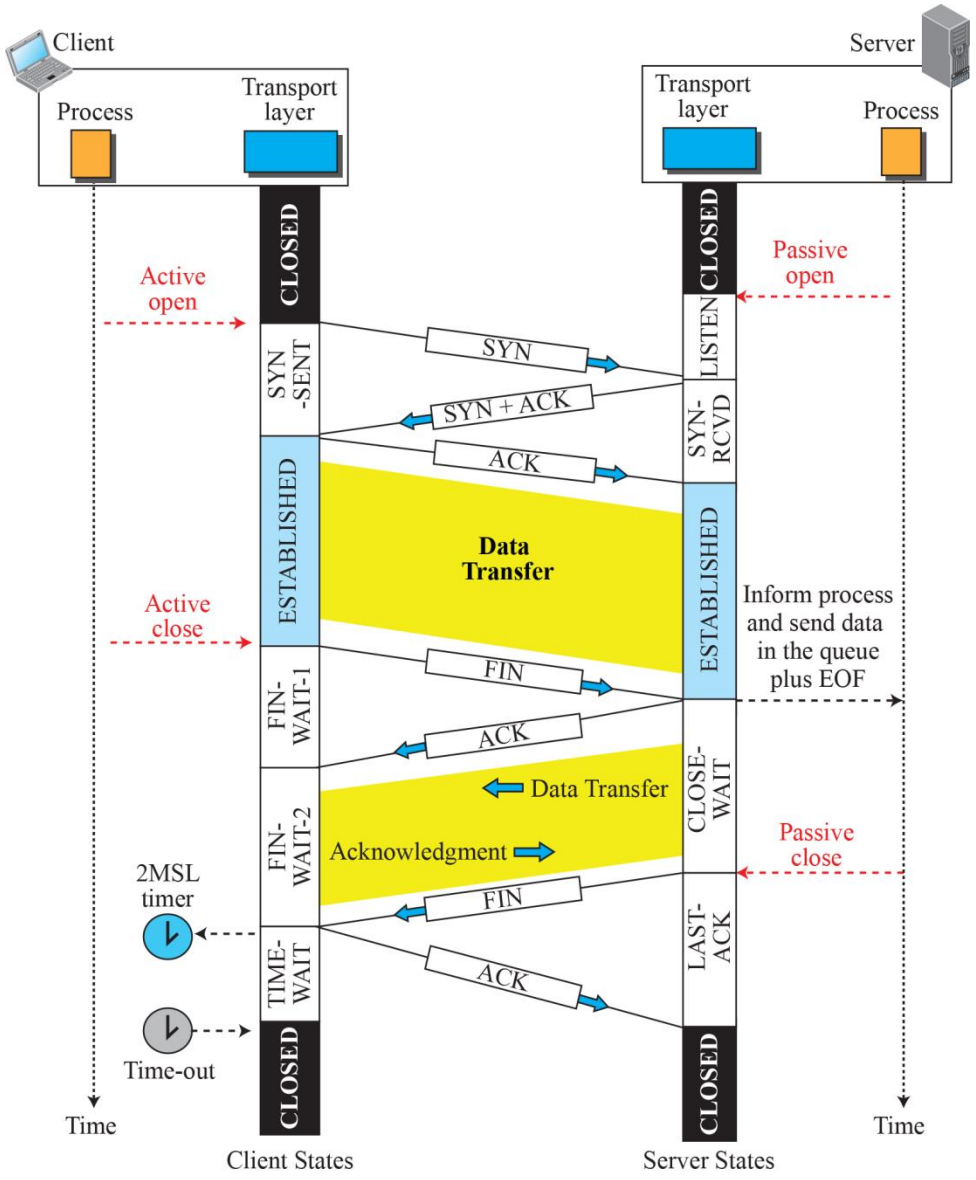
Connection termination using three-way handshaking



Half-close


Client process
Client transport layer
A: ACK flag
F: FIN flag
Server transport layer
Server process
seq: x
ack: y
A
F
FIN
Active close
Connection closed
seq: y – 1
ack: x + 1
A
ACK
Data segments from server to client
Acknowledgment from client to server
seq: z
ack: x + 1
A
F
FIN
Passive close
seq: x + 1
ack: y + 1
A
ACK
Connection closed
Time
Time
Time
Time

Time-line diagram for a common scenario

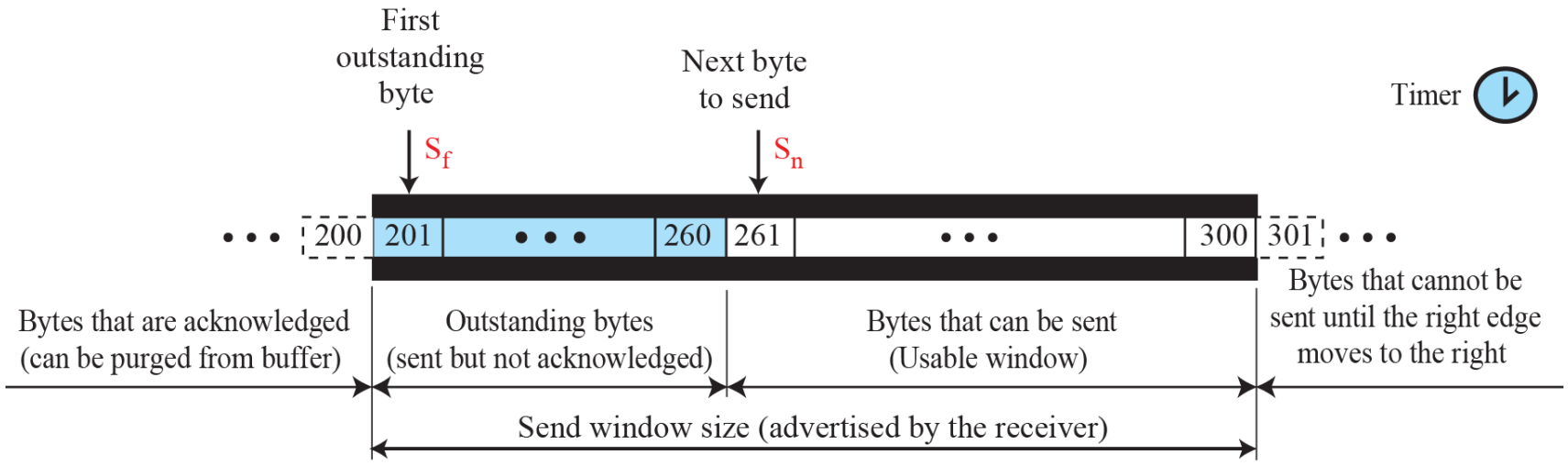


Windows in TCP

Before discussing data transfer in TCP and the issues such as flow, error, and congestion control, we describe the windows used in TCP. TCP uses two windows (send window and receive window) for each direction of data transfer.

we make an unrealistic assumption that communication is only unidirectional. The bidirectional communication can be inferred using two unidirectional communications with piggybacking.

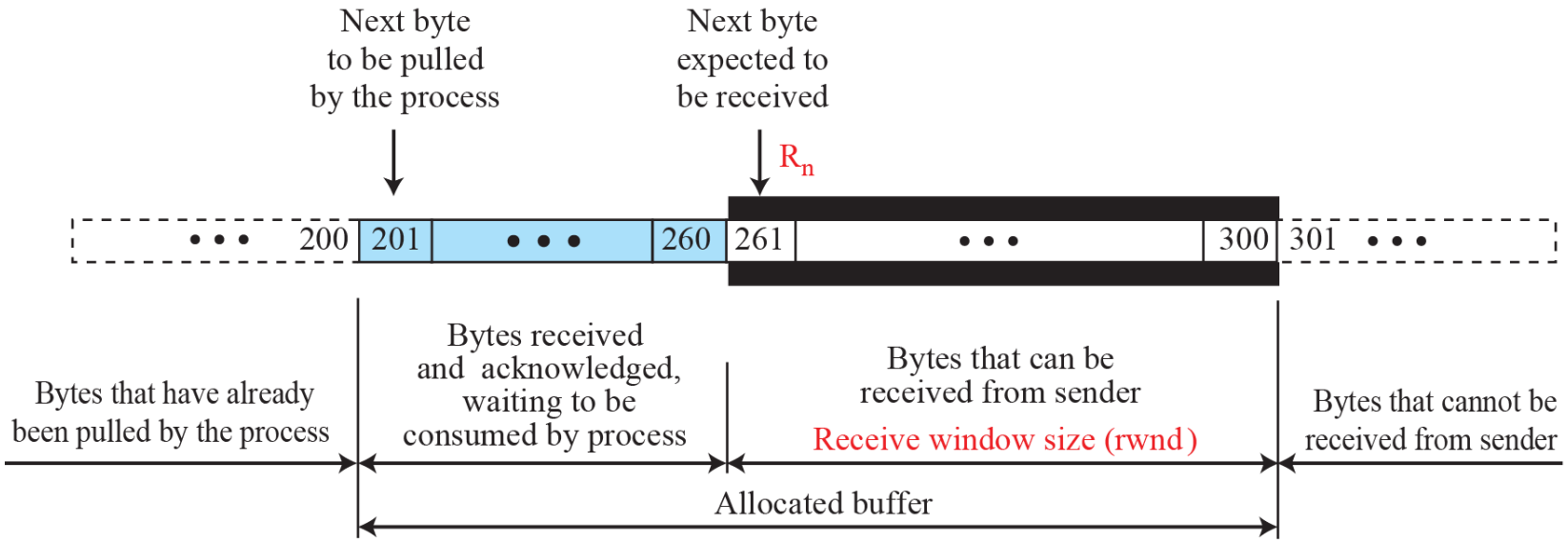
Send window in TCP



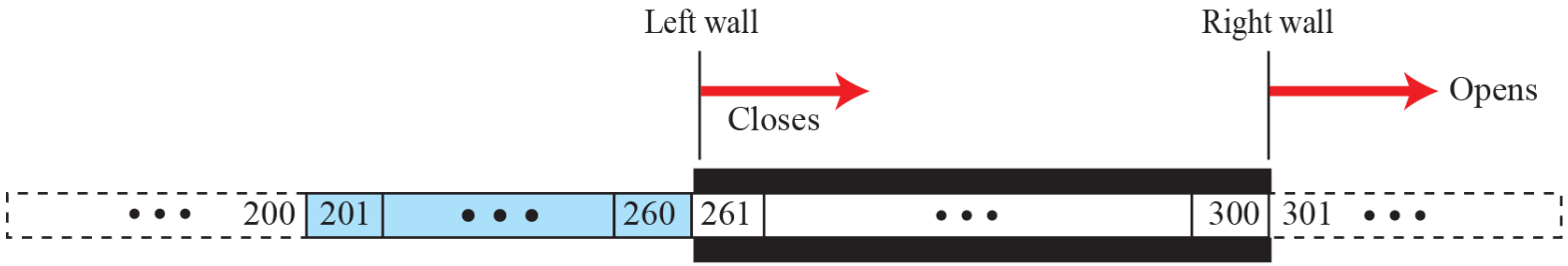
a. Send window

b. Opening, closing, and shrinking send window

Receive window in TCP



a. Receive window and allocated buffer



b. Opening and closing of receive window

Example

What is the value of the receiver window (rwnd) for host A if the receiver, host B, has a buffer size of 5000 bytes and 1000 bytes of received and unprocessed data?

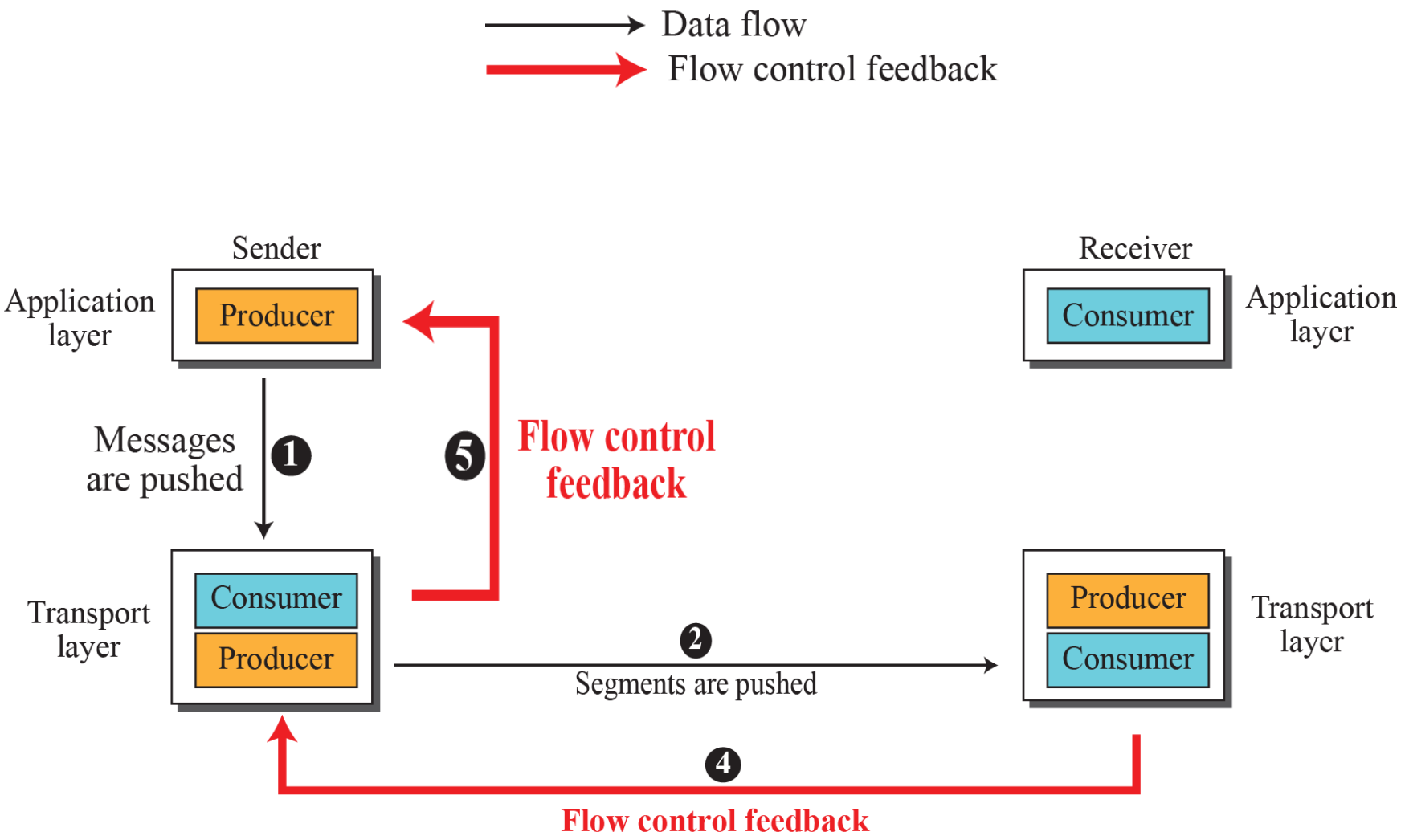
Solution

The value of rwnd = 5000 − 1000 = 4000. Host B can receive only 4000 bytes of data before overflowing its buffer. Host B advertises this value in its next segment to A.

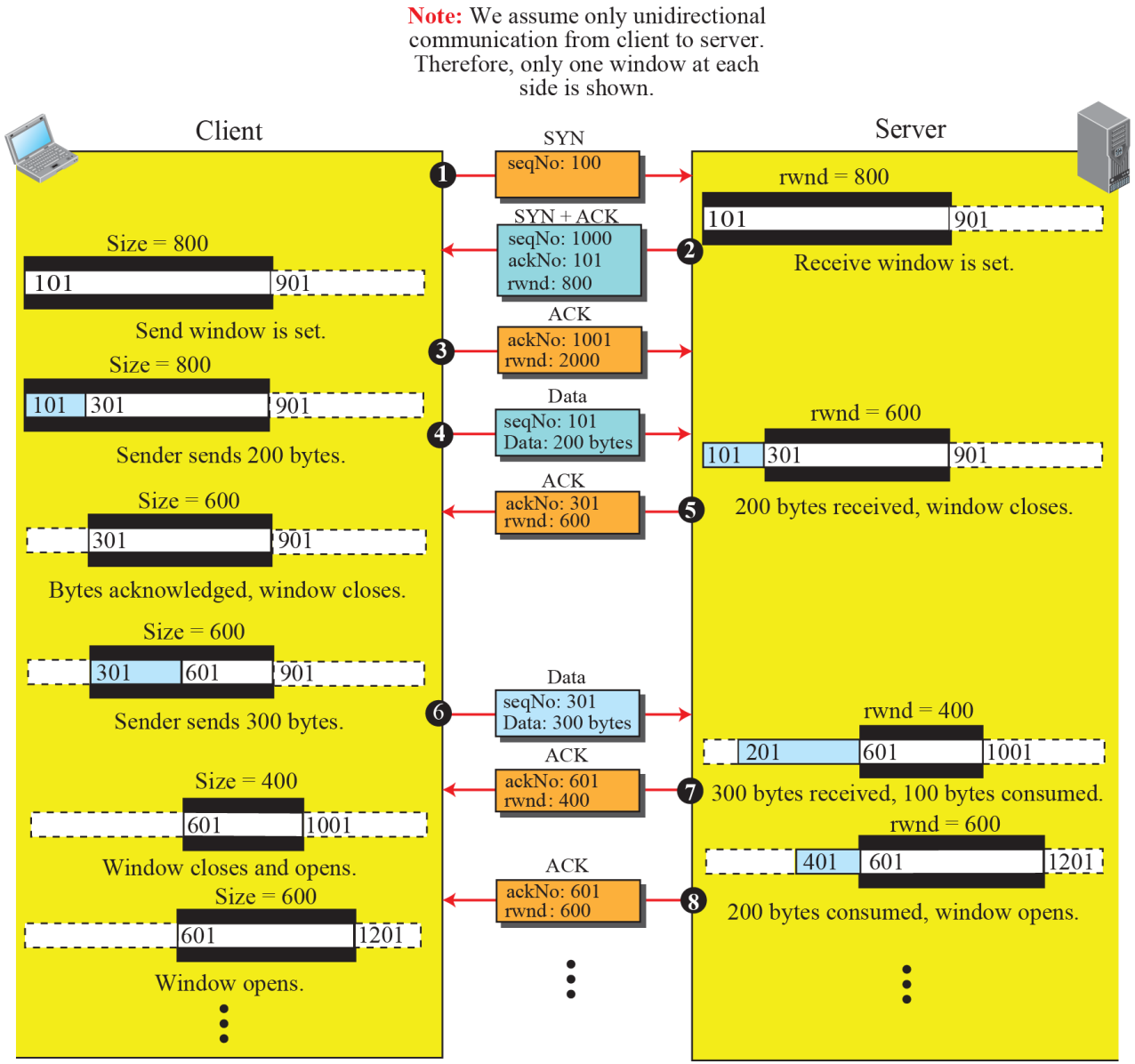
Flow Control

flow control balances the rate a producer creates data with the rate a consumer can use the data. TCP separates flow control from error control. In this section we discuss flow control, ignoring error control. We assume that the logical channel between the sending and receiving TCP is error-free.

Data flow and flow control feedbacks in TCP



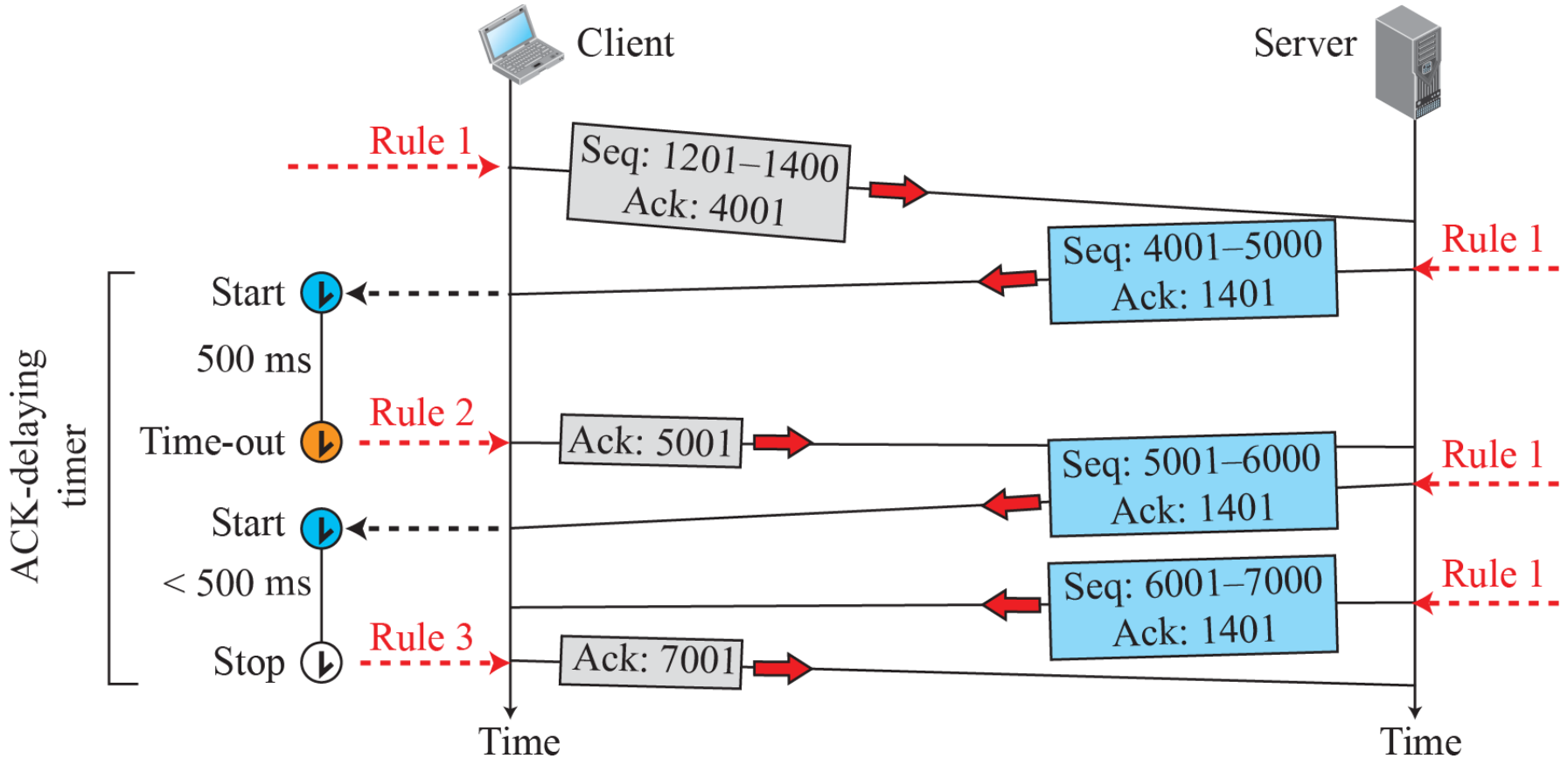
An example of flow control



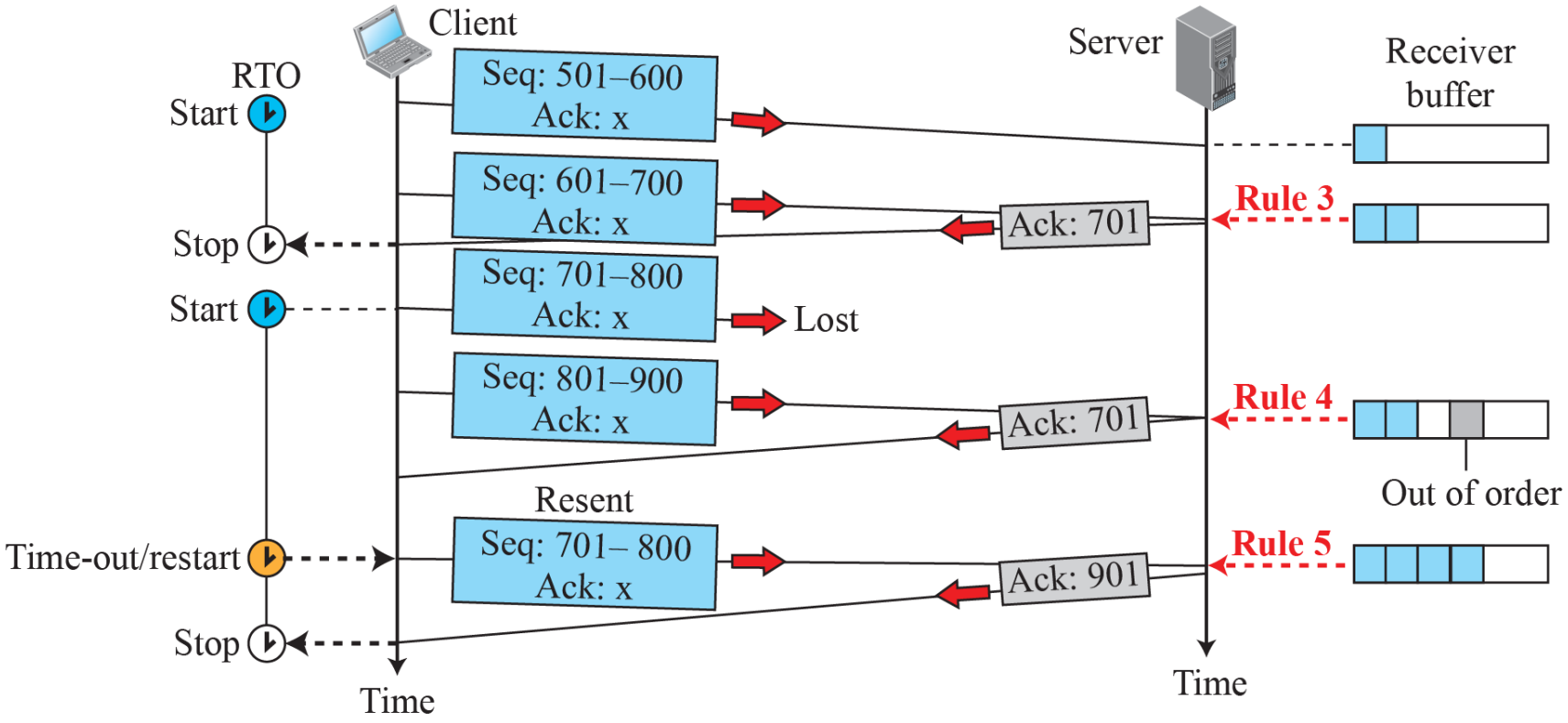
Error Control

TCP is a reliable transport-layer protocol. This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.

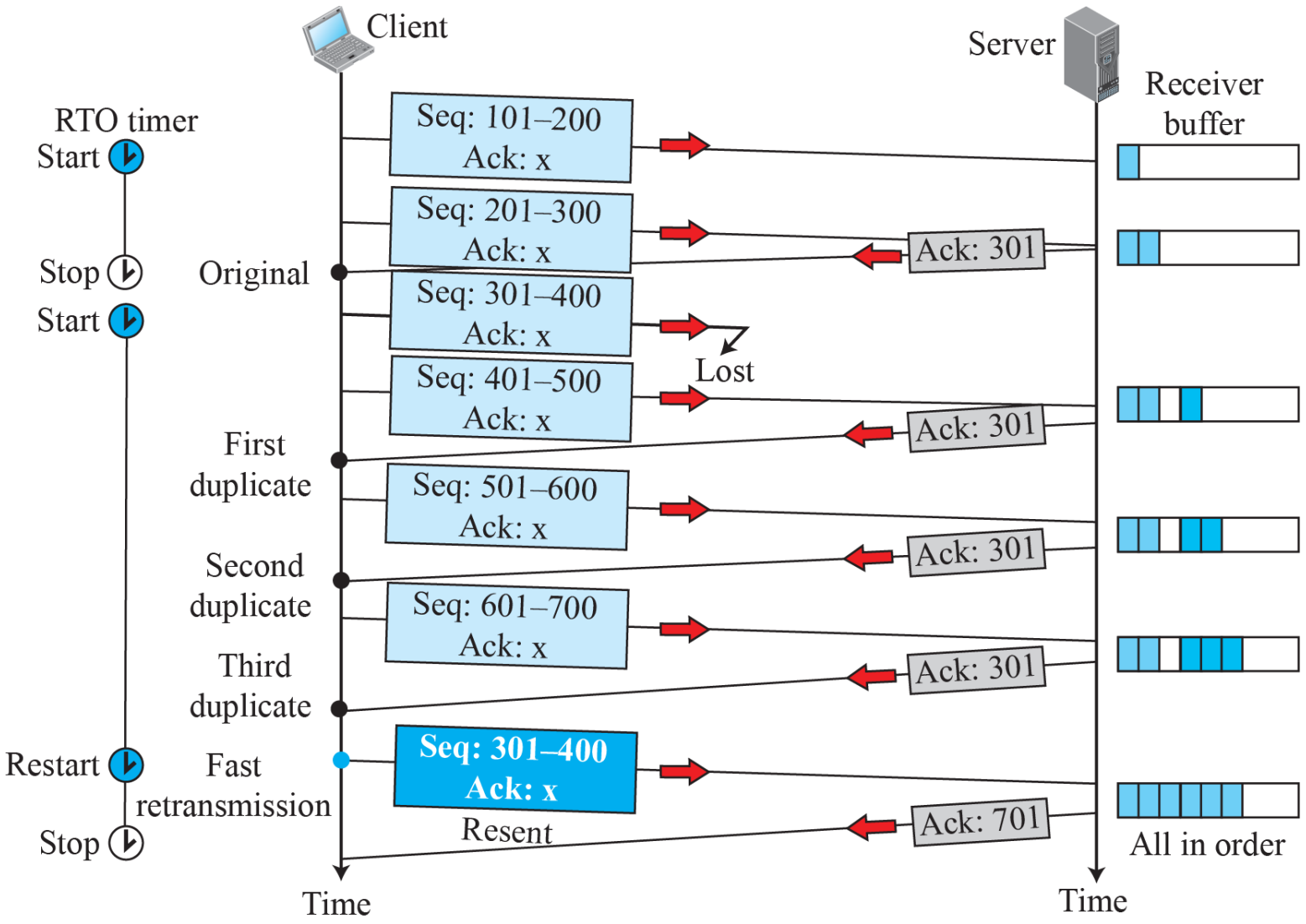
Normal operation



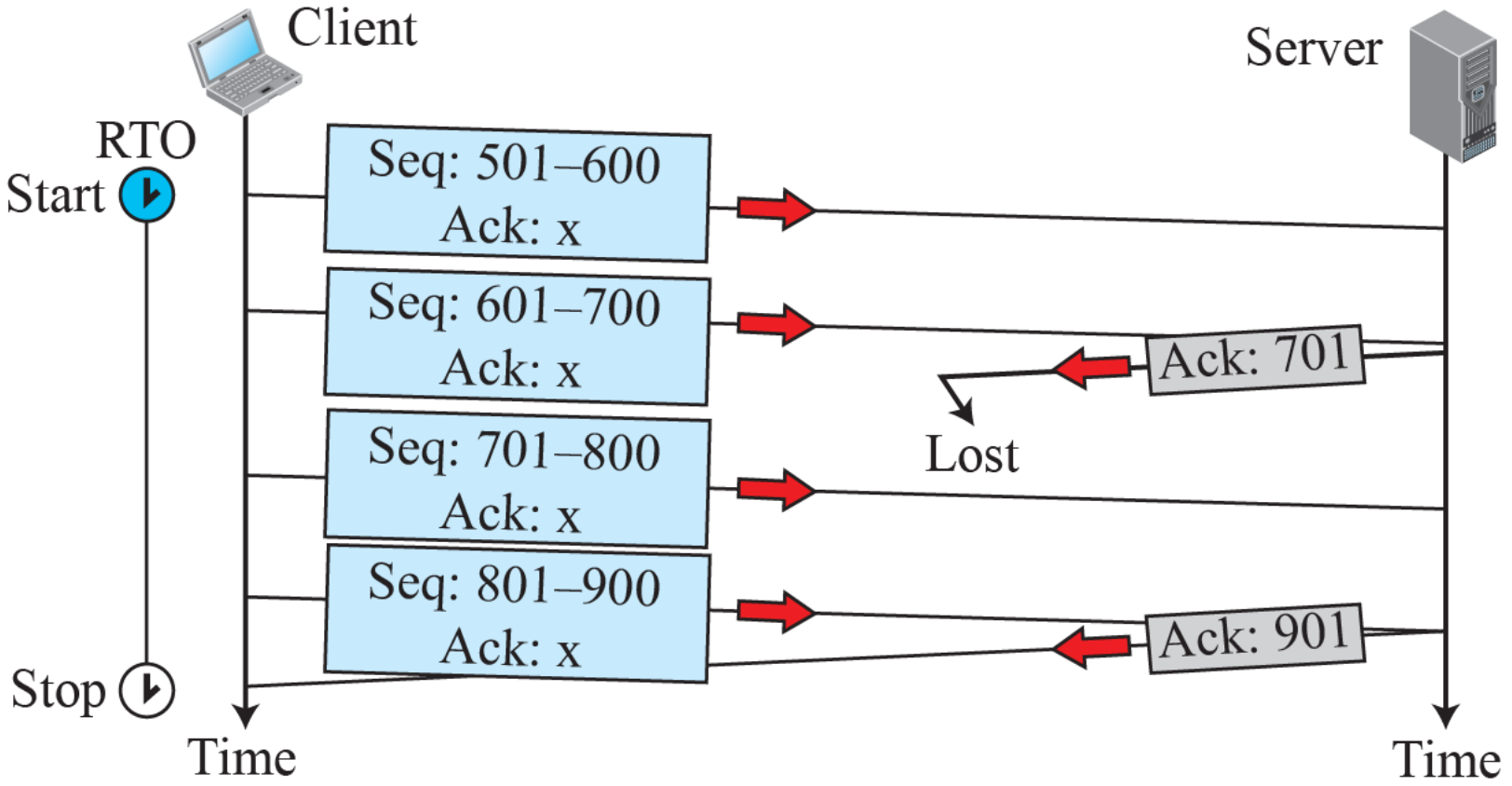
Lost segment



Fast retransmission



Lost acknowledgment



TCP Congestion Control

TCP uses different policies to handle the congestion in the network.

- Flow control in TCP – the size of the send window is controlled by the receiver using the value of *rwnd*,
- To control the number of segments to transmit, TCP uses another variable called a *congestion window, cwnd,* whose size is controlled by the congestion situation in the network.
- The *cwnd* variable and the *rwnd* variable together define the size of the send window in TCP.

Actual window size = **minimum (*rwnd*, *cwnd*)**

Congestion Detection - how a TCP sender can detect the possible existence of congestion

- uses the occurrence of two events as signs of congestion in the network: time-out and receiving three duplicate ACKs.
- four ACKs with the same acknowledgment number.
- when a TCP receiver sends a duplicate ACK, it is the sign that a segment has been delayed, but sending three duplicate ACKs is the sign of a missing segment, which can be due to congestion in the network

Congestion Policies - handling congestion is based on three algorithms: slow start, congestion avoidance, and fast recovery.

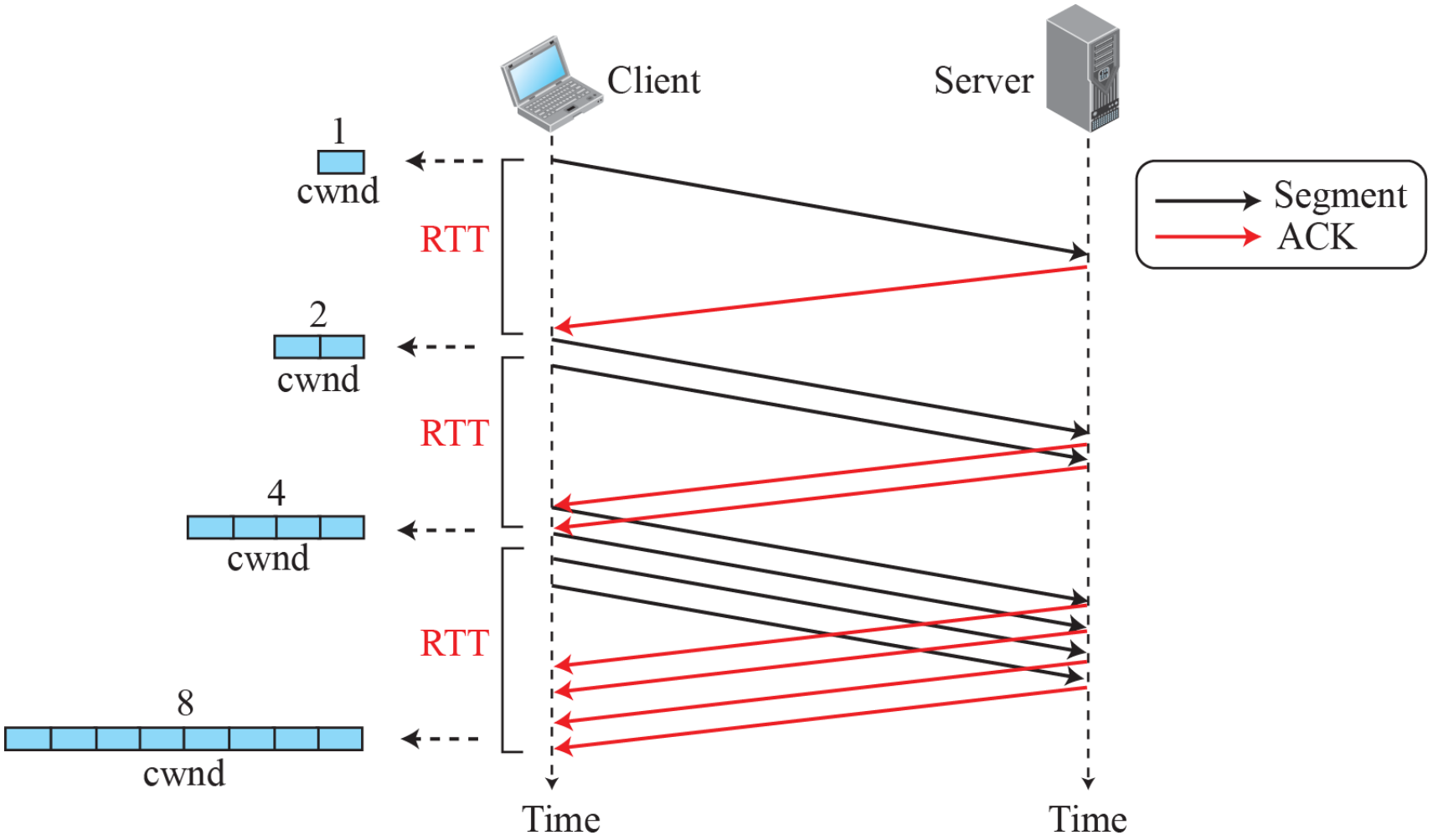
Slow Start: Exponential Increase

is based on the idea that the size of the congestion window (*cwnd*) starts with one maximum segment size (MSS), but it increases one MSS each time an acknowledgment arrives.four ACKs with the same acknowledgment number.

A slow start cannot continue indefinitely. There must be a threshold to stop this phase. The sender keeps track of a variable named *ssthresh* (slow-start threshold).

When the size of the window in bytes reaches this threshold, slow start stops and the next phase starts.

Slow start, exponential increase



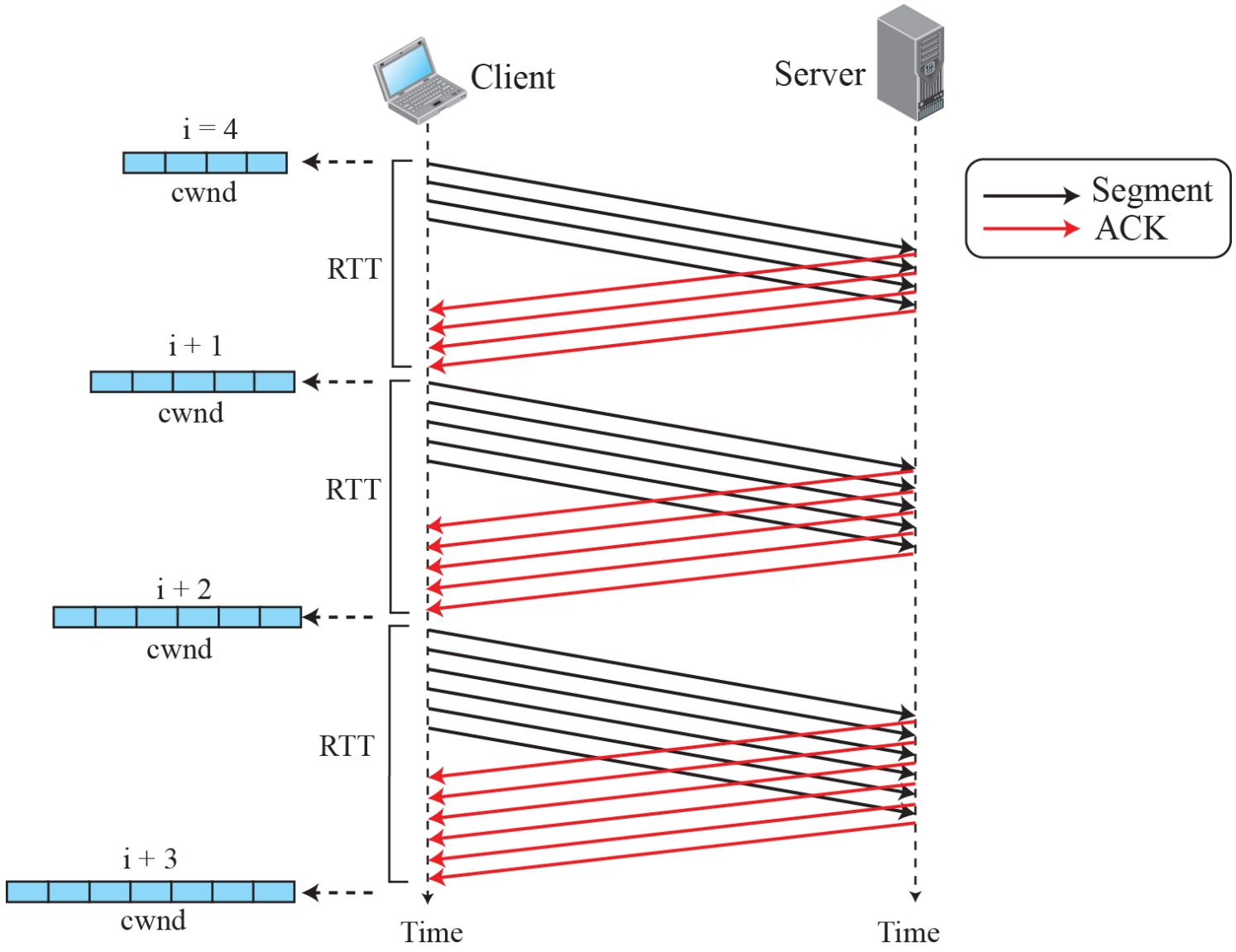
Congestion Avoidance: Additive Increase

the *cwnd* additively instead of exponentially

When the size of the congestion window reaches the slow-start threshold, the slow-start phase stops and the additive phase begins.

all segments in the previous window should be acknowledged to increase the
window 1 MSS bytes

Congestion avoidance, additive increase



Fast Recovery

It starts when three duplicate ACKs arrive,
which is interpreted as light congestion in the network